

Binary Search

Learn how an ordered row lets you rule out half the remaining search positions.

How to use this chapter:

Learn the idea in plain language, read each diagram, predict every trace row, then study the commented Java and attempt the exam-style questions.

Start with the idea

Binary search means checking the middle of an ordered row, then continuing in only the half where the value could still be.

Think of a number-guessing game where the answer is “higher” or “lower”. A middle guess lets you rule out many possibilities at once. This only works for an array if its values are already in order.

A small example

Look for 8 in [2, 4, 6, 8, 10]. The middle value is 6. Because 8 is larger, ignore 6 and everything to its left. In the remaining [8, 10], check 8 and find it. Its original position is **index 3**, counting from 0.

The diagram below searches for 7 in a larger ordered row. “Low” and “high” mark the first and last positions still possible; “mid” is the middle position we check. The values themselves stay still.

The whole process at a glance

Search for 7 in ascending order. The whole run shows inclusive low/high boundaries, the midpoint, and every range ruled out. None of the stored array values move.

Green: sorted / found

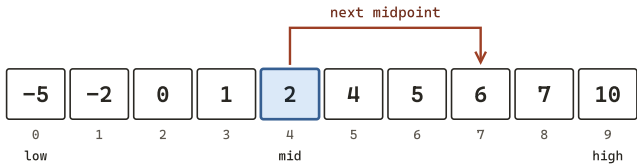
Blue: chosen value / midpoint

Orange: destination / held key

Crossed: ruled out

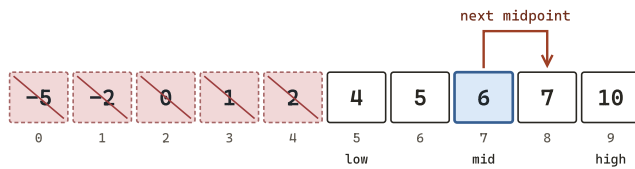
· = conceptual gap

Probe 1



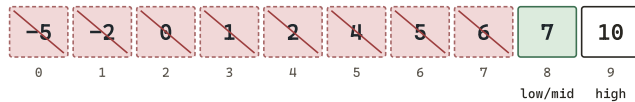
2 < 7: discard indexes 0..4. low = mid + 1 = 5.

Probe 2



6 < 7: discard indexes 5..7. low = mid + 1 = 8.

Probe 3



7 == 7: return index 8. Stop.

Return 8 after 3 midpoint probes. On the successful probe, low = mid = 8 and high = 9. Index 9 remains unvisited because index 8 matches.

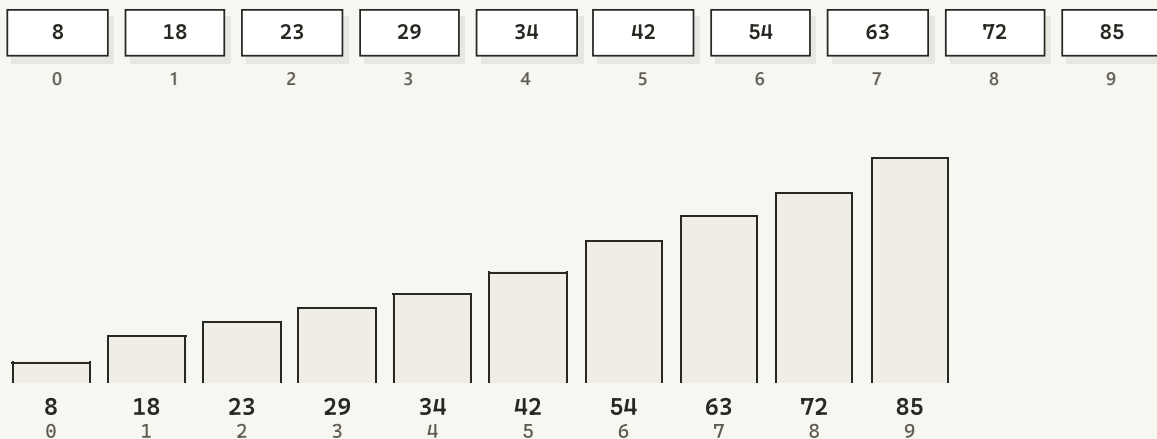
Detailed reference example: this part introduces its own input, shown below. The opening whole-run map and guided walkthrough keep their original input throughout; each later worked example states its new input explicitly.

Searching / Binary search · The array we will search

The array we will search

Ten values, already sorted from smallest to largest: [8, 18, 23, 29, 34, 42, 54, 63, 72, 85, 63, 72, 85].

A SORTED ARRAY OF TEN VALUES



Compare these bars with the random bars in the linear-search chapter. Here the values form a staircase that only goes up. That orderliness is the whole point: **if you look at one cell, the order tells you which side of it the key must be on.** Binary search turns that knowledge into speed.

We will search for 18 (present, at index 1) and for 31 (absent: it falls between 29 and 34).

The textbook definition

Searching / Binary search · The textbook definition

The textbook definition

Binary search finds a value in a *sorted* array by repeatedly comparing the key with the middle element of the part of the array that could still contain it. If the middle element equals the key the search ends. Otherwise the half that cannot contain the key is discarded, and the search continues in the other half. It returns the index of the key, or -1 when no candidates remain.

Precondition: sorted The array must already be in order (ascending, or descending with the comparison flipped). On unsorted data the answer is simply wrong.	low and high The first and last indexes that could still hold the key. Together they are the <i>search window</i> .
mid The probe position, the middle of the window: $\text{low} + (\text{high} - \text{low}) / 2$.	Probe One comparison of the key with <code>arr[mid]</code> .
Discard Throwing away the half that cannot contain the key, by moving <code>low</code> up ($\text{mid} + 1$) or <code>high</code> down ($\text{mid} - 1$).	Not found The window becomes empty: $\text{low} > \text{high}$.
Invariant If the key is in the array at all, it is somewhere in <code>low ... high</code> .	

The name means “two parts”: every probe splits the remaining candidates into two parts and throws one away.

In plain language

Searching / Binary search · In plain language

In plain language

Guess the number I am thinking of by always guessing the middle: each answer throws away half of the possibilities.

You have played this game. I think of a number from 1 to 100, and you guess. After each guess I say “higher” or “lower”. A clever player never guesses randomly: she guesses **50**, the middle. Whatever the answer, half of all the numbers are gone at once. Then she guesses the middle of what is left.

A phone book works the same way. Looking for “Mokoena”, you do not start at page 1: you open the book in the middle, see that you are at “K”, know that M comes later, and ignore everything before that page.

Here is the game for a secret number of **37**:

GUESS NO.	GUESS	ANSWER	NUMBERS STILL POSSIBLE
1	50	too high → guess lower	100
2	25	too low → guess higher	49
3	37	equal: found!	24

Only **3 guesses** were needed out of 100 possibilities. The very worst case for 1 to 100 is 7 guesses, because $100 \rightarrow 50 \rightarrow 25 \rightarrow 12 \rightarrow 6 \rightarrow 3 \rightarrow 1$ uses at most seven halvings. A player who guesses 1, 2, 3, ... needs up to 100.

Binary search is exactly this game, played on an array: `mid` is the guess, “higher / lower” is the comparison with `arr[mid]`, and the sorted order is what lets a single comparison eliminate half of the array.

Searching / Binary search · The main idea

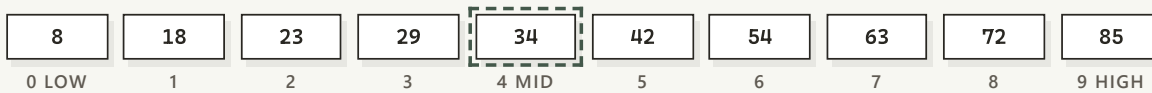
The main idea

Binary search keeps a *window* of cells where the key could still be, and each probe shrinks it. The window starts as the whole array and obeys one rule that never breaks:

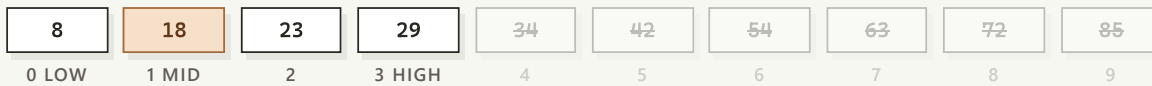
Invariant: if the key is in the array, it is inside the window `low ... high`.

Here is the search for **18**. Crossed-out cells have been discarded; `low`, `mid` and `high` are labelled. After the first probe, **34** is larger than the key, so everything from 34 rightwards is discarded. The second probe lands on **18**.

Probe 1: low = 0, high = 9, mid = 4; arr[mid] = 34 (34 > 18) → search the left half

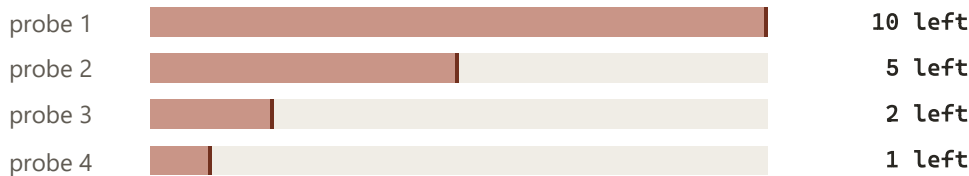


Probe 2: low = 0, high = 3, mid = 1; arr[mid] = 18 (equal) → found it!



Why the middle?

Probing the middle guarantees that, whatever the comparison says, **at least half** the window disappears. Probing anywhere else risks removing less. The window for ten values shrinks roughly $10 \rightarrow 5 \rightarrow 2 \rightarrow 1$, halving each time, so the number of probes is the number of times you can halve n , about $\log_2 n$.



Why the array must be sorted

The discarding step is only legal because of order. “34 > 18, so 18 cannot be to the right of 34” is true *only* if everything to the right of 34 is at least 34. In an unsorted array that statement is false, and binary search would throw away the very cell it was looking for. The mistakes section shows this with a real example.

1

Set low = 0, high = n - 1.

2

While low ≤ high: compute mid.

3

Equal to the key? Return mid.

4

Key larger? low = mid + 1. Key smaller? high = mid - 1.

5

Window empty? Return -1.

Searching / Binary search · Probe, compare, discard

Probe, compare, discard

Each loop turn of binary search is three small steps. Keeping them separate makes the trace table easy to write.

Probe Compute <code>mid</code> and read <code>arr[mid]</code> .	Compare Three possible answers: <code>arr[mid] == key</code> (found), <code>arr[mid] < key</code> (key is to the right), <code>arr[mid] > key</code> (key is to the left).
Discard Move one end of the window past <code>mid</code> : <code>low = mid + 1</code> or <code>high = mid - 1</code> .	Iteration One probe, one comparison outcome and one discard. "After how many iterations?" counts these.

Why `mid + 1` and `mid - 1`, not just `mid`?

The probe at `mid` has already been compared and failed, so `mid` itself is ruled out. If you wrote `low = mid` the window could stay the same size forever, for example with `low = 2`, `high = 3`, where `mid` is 2. Stepping past `mid` guarantees the window shrinks every time, so the loop must end.

Why `low + (high - low) / 2` and not `(low + high) / 2`?

For small arrays they give the same answer. But `low + high` can be too big for an `int`. An `int` in Java holds numbers up to 2,147,483,647 (about 2.1 billion). With `low = 2,000,000,000` and `high = 2,100,000,000`:

FORMULA	CALCULATION	RESULT
<code>(low + high) / 2</code>	<code>2,000,000,000 + 2,100,000,000 = 4,100,000,000</code> : more than the largest <code>int</code> , so Java <i>wraps round</i> to a negative number	<code>-194,967,296</code> → <code>-97,483,648</code> (a negative index: crash)
<code>low + (high - low) / 2</code>	<code>2,100,000,000 - 2,000,000,000 = 100,000,000</code> , halved is <code>50,000,000</code> , plus <code>low</code>	<code>2,050,000,000</code> (correct)

The second form only ever adds half of the *distance* between the ends to `low`, so it can never leave the range of valid indexes. The extra effort costs nothing, so it is the form to always use.

Reading a trace

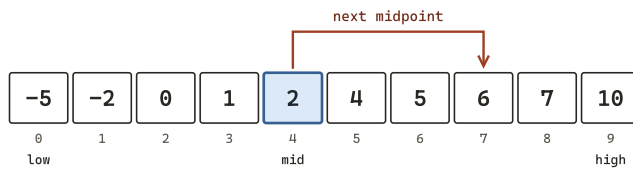
In a trace table, the *Candidates* column is the number of cells in the window, `high - low + 1`. It roughly halves down each row until it reaches 0 (not found) or the key is hit.

Guided walkthrough: predict the next major step

Predict the next completed check, then advance. Each scene keeps all its movements together.

Guided walkthrough: major steps

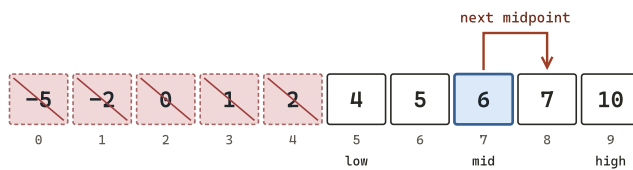
Probe 1



$2 < 7$: discard indexes 0..4. $\text{low} = \text{mid} + 1 = 5$.

$2 < 7$: discard indexes 0..4. $\text{low} = \text{mid} + 1 = 5$.

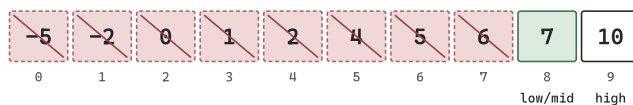
Probe 2



$6 < 7$: discard indexes 5..7. $\text{low} = \text{mid} + 1 = 8$.

$6 < 7$: discard indexes 5..7. $\text{low} = \text{mid} + 1 = 8$.

Probe 3



$7 == 7$: return index 8. Stop.

$7 == 7$: return index 8. Stop.

▼ Full trace table - select a row to jump to its scene

1	0	4	9	2	$2 < 7$: discard indexes 0..4. $\text{low} = \text{mid} + 1 = 5$.
2	5	7	9	6	$6 < 7$: discard indexes 5..7. $\text{low} = \text{mid} + 1 = 8$.
	8	8	9	7	$7 == 7$: return index 8. Stop.
PROBE	LOW	MID	HIGH	VALUE	DECISION

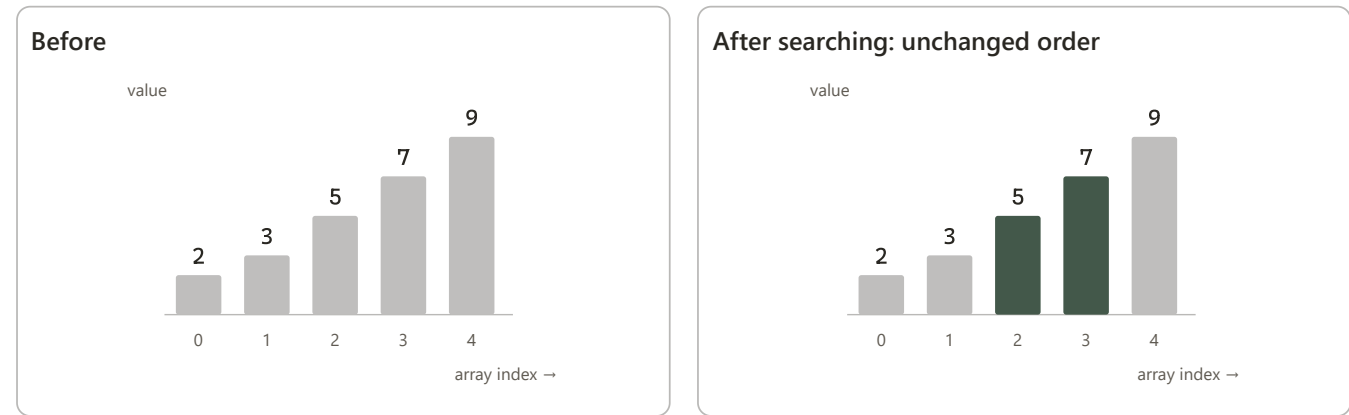
The trace at a glance

PROBE	LOW	MID	HIGH	VALUE	DECISION
1	0	4	9	2	$2 < 7$: discard indexes 0..4. $\text{low} = \text{mid} + 1 = 5$.
2	5	7	9	6	$6 < 7$: discard indexes 5..7. $\text{low} = \text{mid} + 1 = 8$.
3	8	8	9	7	$7 == 7$: return index 8. Stop.

Read the algorithm as a bar graph

Bar height means **value**; horizontal position means **index**. All panels below use the same height scale. A taller bar is not "more sorted".

Order lets a probe rule out a whole side



Searching for 7 first probes index 2, value 5. Because $5 < 7$ and the bars rise left to right, indexes 0..2 cannot contain 7. Set $\text{low} = 3$; the next midpoint is 3 and matches. No bar moves: only the candidate interval changes.

Check yourself: identify which indexes are guaranteed final, which values were merely inspected, and whether the operation swapped, shifted or copied data. A sorted ascending array has non-decreasing heights, including equal-height neighbours.

Searching / Binary search

Discard half each time

Binary search works only on a sorted array. Compare with the middle: go left when the key is smaller, right when it is larger, or stop when equal.

Non-negotiable rule: if the array is not sorted by the same key, binary search can discard the part that actually contains the answer.

Illustration



Searching for the highlighted 18. First compare with 34 at index 4 (the 1st midpoint). Because 18 is smaller, indexes 4 to 9 are discarded. The 2nd midpoint is index 1, where 18 is found.

Trace table

STEP	LOW	HIGH	MID	VALUE	ACTION
1	0	9	4	34	18 < 34 → high = 3
2	0	3	1	18	Match → return 1

Implementation

●●● Iterative binary search

```
public static int binarySearch(int[] arr, int key) {
    // The active search range includes both endpoints.
    int low = 0, high = arr.length - 1;
    while (low ≤ high) {
        // This form avoids overflow from (low + high) / 2.
        int mid = low + (high - low) / 2;
        if (arr[mid] == key) return mid;
        // Ascending order: larger keys can only be on the right.
        if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    // The range is empty when low passes high.
    return -1;
}
```

▼ Recursive implementation

Java

```
public static int binarySearchRec(int[] arr, int key,
                                int low, int high) {
    // Base case: no indexes remain.
    if (low > high) return -1;
    int mid = low + (high - low) / 2;
    if (arr[mid] == key) return mid;
    // Recur into exactly one smaller half.
    if (key < arr[mid])
        return binarySearchRec(arr, key, low, mid - 1);
    return binarySearchRec(arr, key, mid + 1, high);
}
```

Write it from scratch

- 1 Set low and high.
- 2 Repeat while $\text{low} \leq \text{high}$.
- 3 Calculate mid.
- 4 Return, move low, or move high.
- 5 Return -1.

Descending-order change

Equality stays the same, but the direction reverses: when $\text{key} > \text{arr}[\text{mid}]$, set $\text{high} = \text{mid} - 1$; otherwise set $\text{low} = \text{mid} + 1$.

Same comparison, opposite decision. In a descending array the larger values are on the left, so the branches swap. Search for 18 in [85, 72, 63, 54, 42, 34, 29, 23, 18, 8]:

STEP	LOW	HIGH	MID	ARR[MID]	DECISION (DESCENDING)
1	0	9	4	42	$18 < 42 \rightarrow$ keys are to the right $\rightarrow \text{low} = 5$
2	5	9	7	23	$18 < 23 \rightarrow$ 8
3	8	9	8	18	Match $\rightarrow$ return

Debug checklist

- Confirm the array's ordering first.
- Use $\text{mid} \pm 1$ so the range shrinks.
- Keep $\text{low} \leq \text{high}$ so a one-cell range is checked.

Worked example · key absent

Search for 31 in the sorted array below. Every row shows the complete array; crossed-out cells are outside the active range.

Step 1 · low = 0, high = 9, mid = 4

arr[4] = 34. Because 31 < 34, discard index 4 and everything to its right: high = 3.

8

18

23

29

34

42

54

63

72

85

0 LOW

1

2

3

4 MID

5

6

7

8

9 HIGH

Step 2 · low = 0, high = 3, mid = 1

arr[1] = 18. Because 31 > 18, discard indexes 0 and 1: low = 2.

8

18

23

29

34

42

54

63

72

85

0 LOW

1 MID

2

3 HIGH

4

5

6

7

8

9

Step 3 · low = 2, high = 3, mid = 2

arr[2] = 23. Because 31 > 23, set low = 3.

8

18

23

29

34

42

54

63

72

85

0

1

2 LOW/MID

3 HIGH

4

5

6

7

8

9

Step 4 · low = high = mid = 3

arr[3] = 29. Because 31 > 29, set low = 4. Now low > high, so no candidate remains.

8

18

23

29

34

42

54

63

72

85

0

1

2

3 ALL

4

5

6

7

8

9

Result: 31 is absent → return -1 after 4 comparisons.

STEP	LOW	HIGH	MID	ARR[MID]	UPDATE
1	0	9	4	34	high = 3
2	0	3	1	18	low = 2
3	2	3	2	23	low = 3
4	3	3	3	29	low = 4

Duplicates · which match is returned?

Binary search returns *whichever equal value the midpoint lands on first*. It is not guaranteed to be the first or the last copy. Search for 20 in [10, 20, 20, 20, 30]:

CSI247 · Binary Search

11 / 36



$\text{mid} = 0 + (4 - 0) / 2 = 2$ and $\text{arr}[2] == 20 \rightarrow$ return 2 after 1 comparison. Index 1 also holds 20, but it is never reached.

If a lab needs the *first* occurrence, keep searching left after a match instead of returning immediately.

BEST

$O(1)$

AVERAGE

$O(\log n)$

WORST

$O(\log n)$

SORTED?

Required

ITERATIVE SPACE

$O(1)$

RECURSIVE SPACE

$O(\log n)$

DUPLICATES

Any match

INPUT CHANGED?

No

Questions

▼ Why is binary search faster on large arrays?

Each comparison discards about half of the remaining values, so the number of steps grows logarithmically rather than linearly.

▼ What is the recursive base case for “not found”?

$\text{low} > \text{high}$. It means no indexes remain in the current search range.

Binary search in one sentence

On a correctly sorted array, compare with the middle and discard the impossible half until a match is found or the range is empty.

Fully annotated Java: connect each line to the movement

```
java

public static int binarySearch(int[] arr, int key) {
    // PRECONDITION: arr is in ascending order by the searched value.
    int low = 0, high = arr.length - 1;
    // Inclusive endpoints: a one-element range must still be checked.
    while (low ≤ high) {
        // Overflow-safe midpoint; integer division rounds down here.
        int mid = low + (high - low) / 2;
        if (arr[mid] == key) return mid;
        if (arr[mid] < key) {
            // Mid was already checked; exclude it from the next range.
            low = mid + 1;
        } else {
            // All values from mid rightwards are too large.
            high = mid - 1;
        }
    }
    // low > high means no candidate index remains.
    return -1;
}
```

Build it from scratch

- State the sorted-input precondition.
- Initialize the inclusive range.
- Repeat while a candidate remains.
- Calculate the midpoint; test equality.
- Exclude mid and the impossible side.
- Return -1 after the range becomes empty.

Recursion, calls and returns

java

```
public static int binarySearchRec(int[] arr, int key, int low, int high) {  
    // Empty range must return before calculating/accessing mid.  
    if (low > high) return -1;  
    int mid = low + (high - low) / 2;  
    if (arr[mid] == key) return mid;  
    // Exactly one recursive child is needed; the other half is impossible.  
    if (key < arr[mid]) return binarySearchRec(arr, key, low, mid - 1);  
    return binarySearchRec(arr, key, mid + 1, high);  
    // First call: binarySearchRec(arr, key, 0, arr.length - 1).  
}
```

CALL STACK - NEWEST FRAME AT THE TOP

binarySearchRec(0, 3)

mid 1 matches 18: return 1

binarySearchRec(0, 9)

Waiting: Return the result from child range 0..3

The matching child returns 1; its parent forwards that result. Only one branch is called.

Initial call: `binarySearchRec(arr, key, 0, arr.length - 1)`. Depth is $O(\log n)$ because each child retains at most about half the range.

A complete runnable Java example

Save as `BinaryDemo.java`, compile with `javac`, then run the class name without an extension.

```
import java.util.Arrays;

public class BinaryDemo {
    public static int binarySearch(int[] arr, int key) {
        // PRECONDITION: arr is in ascending order by the searched value.
        int low = 0, high = arr.length - 1;
        // Inclusive endpoints: a one-element range must still be checked.
        while (low ≤ high) {
            // Overflow-safe midpoint; integer division rounds down here.
            int mid = low + (high - low) / 2;
            if (arr[mid] == key) return mid;
            if (arr[mid] < key) {
                // Mid was already checked; exclude it from the next range.
                low = mid + 1;
            } else {
                // All values from mid rightwards are too large.
                high = mid - 1;
            }
        }
        // low > high means no candidate index remains.
        return -1;
    }

    public static void main(String[] args) {
        int[] values = {8, 18, 23, 29, 34, 42, 54, 63, 72, 85};
        System.out.println(binarySearch(values, 18));
    }
}
```

1

Searching / Binary search · Properties

Properties

BEST TIME $O(1)$	AVERAGE TIME $O(\log n)$
WORST TIME $O(\log n)$	EXTRA SPACE $O(1)^*$
NEEDS SORTED DATA Yes	NEEDS RANDOM ACCESS Yes
COMPARES WITH $<, >, ==$	FINDS Any one match

* iterative version. The recursive version uses $O(\log n)$ stack space.

Sorted data only

This is the one big requirement. If the data is not sorted, either sort it first ($O(n \log n)$) or use linear search.

Random access

It jumps straight to `arr[mid]`. That is instant in an array but slow in a linked list, where reaching the middle means walking there. On a linked list, binary search loses its advantage.

Duplicates

If the key appears several times, binary search returns *one* of them, not necessarily the first. With `[10, 20, 20, 20, 30]` and key 20 it returns index 2, the middle one. Finding the first or last occurrence needs a small variation.

Logarithmic cost

Doubling the array size adds only **one** extra probe.

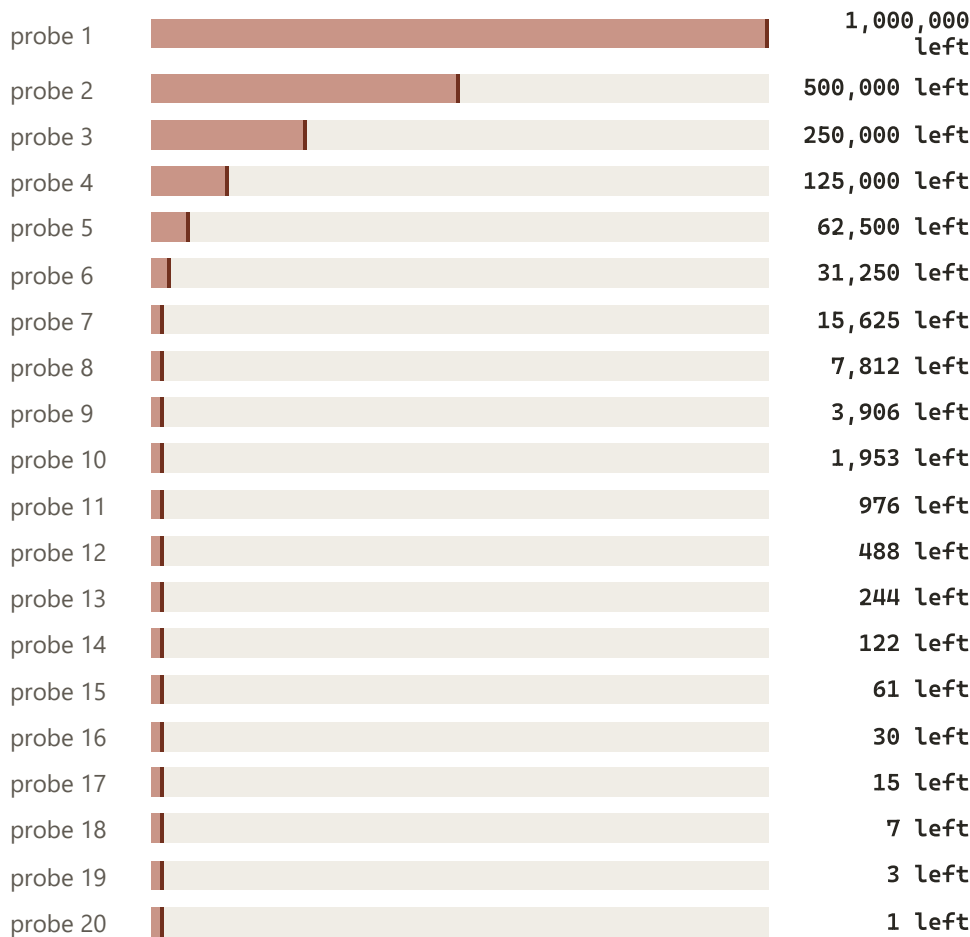
Searching / Binary search · Why those complexities occur

Why those complexities occur

Best case · $O(1)$ The key is the middle cell on the first probe.	Worst case · $O(\log n)$ The window halves each probe, so at most $\lfloor \log_2 n \rfloor + 1$ probes.
--	---

Halving a million

For a million values, here is how many candidates remain after each probe:



Twenty probes take you from a million candidates to one. That is the whole story of $O(\log n)$: **how many times can you halve n before you reach 1?**

$$\text{maximum probes} = \lfloor \log_2 n \rfloor + 1$$

N (VALUES)	BINARY SEARCH: AT MOST	LINEAR SEARCH: AT MOST	BINARY IS ABOUT THIS MANY TIMES FEWER
10	4	10	$\approx 2\times$
100	7	100	$\approx 14\times$
1,000	10	1,000	$\approx 100\times$
1,000,000	20	1,000,000	$\approx 50,000\times$
1,000,000,000	30	1,000,000,000	$\approx 33,333,333\times$

Ten times more data adds about 3 probes, not 10 times the work. Even for a *billion* values binary search needs at most 30 probes.

Average and best case

The average is still about $\log_2 n$: most keys sit deep in the tree of halving, not near the first probe. The best case is a single probe.

Is it worth sorting first?

Binary search is fast, but it needs sorted data, and sorting costs about $n \log_2 n$. For $n = 1,000,000$ that is roughly **20,000,000** steps, once. Each later linear search costs up to 1,000,000 steps but each binary search only 20, saving almost a million steps per search. So the sort pays for itself after about **21 searches**. For one search, just scan; for many, sort once.

Space

The iterative version needs just `low`, `high` and `mid`: $O(1)$.

Searching / Binary search · When to use it

When to use it

A good choice Large collections that are sorted (or that you can sort once) and searched many times; dictionaries, phone books, product catalogues, leaderboards; any array with random access.	A poor choice Unsorted data searched once; very small arrays (linear search is just as fast and simpler); linked lists (no fast access to the middle); data that changes constantly, since keeping it sorted costs time.
---	--

The decision rule: *sorted + random access + searched more than a few times* → *binary search*. Otherwise linear search. This is exactly the choice behind the past paper's last question: of the four algorithms listed, binary search is the most efficient for searching, $O(\log n)$.

Searching

What searching means

Searching is the process of looking for a particular element in a collection. A successful method returns its index; an unsuccessful search returns **-1**.

Search key The value being looked for.	Comparison Checking the key against an array value.
Ordered collection Values already arranged by a rule.	Unordered collection Values may appear in any order.

Use a search to check membership, display or update an item, or find the position needed for another operation.

Reading Big-O: why we count steps

Big-O describes how the number of steps grows as the number of values n grows. It ignores small constant details and keeps the dominant growth. *Best*, *average* and *worst* case describe three situations for the same size n : the most favourable input, a typical input, and the least favourable input. In this guide one *comparison* means one probe: one look at an array value (for binary search, `arr[mid]`), even though Java may spell it as two tests such as `==` then `<`.

N VALUES	BINARY SEARCH $O(\log N)$ WORST COMPARISONS	LINEAR SEARCH $O(N)$	MERGE SORT $O(N \log N)$	SIMPLE SORTS $O(N^2)$ WORST
10	4	10	≈ 33	100
100	7	100	≈ 664	10,000
1,000	10	1,000	$\approx 9,966$	1,000,000
1,000,000	20	1,000,000	≈ 20 million	1,000,000,000,000

Binary search column: $\lceil \log_2 n \rceil + 1$ comparisons. Merge column: $n \times \log_2 n$. These are step counts, not seconds.

$O(1)$ · constant The work does not depend on n , such as reading <code>marks[0]</code> .	$O(\log n)$ · logarithmic Each step discards a fraction of the data. Doubling n adds one step.
$O(n)$ · linear Each value is touched about once. Doubling n doubles the work.	$O(n^2)$ · quadratic Each value may meet every other value. Doubling n quadruples the work.

Searching / Binary search · Descending order

Descending order

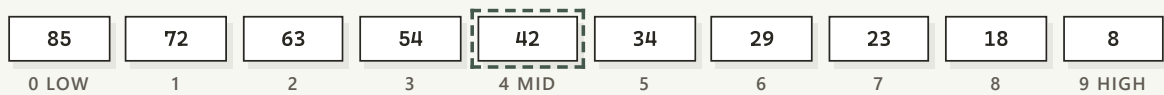
If the array is sorted from *largest* to *smallest*, the logic is the same but the directions swap: a probe that is *larger* than the key means the key is to the **right**, where the smaller values are.

Ascending array	Descending array
<pre>if (arr[mid] == key) return mid; if (arr[mid] < key) low = mid + 1; else high = mid - 1;</pre>	<pre>if (arr[mid] == key) return mid; if (arr[mid] > key) low = mid + 1; else high = mid - 1;</pre>

Only the single comparison changes. Searching for 18 in the descending array [85, 72, 63, 54, 42, 34, 29, 23, 18, 8]:

85	72	63	54	42	34	29	23	18	8
0	1	2	3	4	5	6	7	8	9

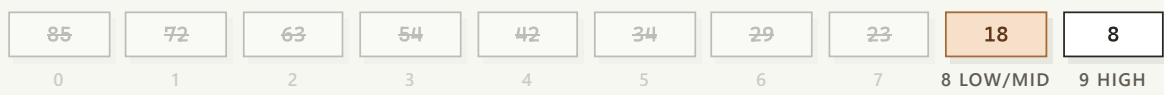
Probe 1: low = 0, high = 9, mid = 4; arr[mid] = 42 (42 > 18) → search the right half



Probe 2: low = 5, high = 9, mid = 7; arr[mid] = 23 (23 > 18) → search the right half



Probe 3: low = 8, high = 9, mid = 8; arr[mid] = 18 (equal) → found it!



PROBE	LOW	HIGH	MID	ARR[MID]	QUESTION	DECISION	CANDIDATES
1	0	9	$0 + (9 - 0) / 2 = 4$	42	$42 > 18$	search the right half	10
2	5	9	$5 + (9 - 5) / 2 = 7$	23	$23 > 18$	search the right half	5
3	8	9	$8 + (9 - 8) / 2 = 8$	18	$18 == 18$	found	2

Found at index 8 after 3 probes. The count is the same kind of count as the ascending search: the order of the data changes the *direction* of each decision, never the number of halvings.

Test habit: before tracing, look at the array and decide which way it is sorted, then write down what “larger than the key” means for that array.

Searching / Binary search · Worked examples

Worked examples

Six searches: the best case, a key at the far end, keys outside the range, a key between two values, and question 3(c) from the past paper.

Found at the first probe · search 34

The middle cell happens to be the key.



Probe 1: low = 0, high = 9, mid = 4; arr[mid] = 34 (equal) → found it!

8	18	23	29	34	42	54	63	72	85
0 LOW	1	2	3	4 MID	5	6	7	8	9 HIGH

PROBE	LOW	HIGH	MID	ARR[MID]	QUESTION	DECISION	CANDIDATES
1	0	9	$0 + (9 - 0) / 2 = 4$	34	$34 == 34$	found	10

1 probe, returns 4. This is the best case: $O(1)$.

Found at the very end · search 85

The key is the last cell, so the window must shrink all the way to it.

8	18	23	29	34	42	54	63	72	85
0	1	2	3	4	5	6	7	8	9

Probe 1: low = 0, high = 9, mid = 4; arr[mid] = 34 ($34 < 85$) → search the right half

8	18	23	29	34	42	54	63	72	85
0 LOW	1	2	3	4 MID	5	6	7	8	9 HIGH

Probe 2: low = 5, high = 9, mid = 7; arr[mid] = 63 ($63 < 85$) → search the right half

8	18	23	29	34	42	54	63	72	85
0	1	2	3	4	5 LOW	6	7 MID	8	9 HIGH

Probe 3: low = 8, high = 9, mid = 8; arr[mid] = 72 ($72 < 85$) → search the right half

8	18	23	29	34	42	54	63	72	85
0	1	2	3	4	5	6	7	8 LOW/MID	9 HIGH

Probe 4: low = 9, high = 9, mid = 9; arr[mid] = 85 (equal) → found it!

8	18	23	29	34	42	54	63	72	85
0	1	2	3	4	5	6	7	8	9 LOW/MID/HIGH

PROBE	LOW	HIGH	MID	ARR[MID]	QUESTION	DECISION	CANDIDATES
1	0	9	$0 + (9 - 0) / 2 = 4$	34	$34 < 85$	search the right half	10
2	5	9	$5 + (9 - 5) / 2 = 7$	63	$63 < 85$	search the right half	5
3	8	9	$8 + (9 - 8) / 2 = 8$	72	$72 < 85$	search the right half	2
4	9	9	$9 + (9 - 9) / 2 = 9$	85	$85 == 85$	found	1

4 probes, returns 9. A key at either end is a near-worst case, but still only 4 probes for 10 values.

Smaller than everything · search 5

The key is below the smallest value, so every probe says "go left".

8	18	23	29	34	42	54	63	72	85
0	1	2	3	4	5	6	7	8	9

Probe 1: low = 0, high = 9, mid = 4; arr[mid] = 34 ($34 > 5$) → search the left half

8	18	23	29	34	42	54	63	72	85
0 LOW	1	2	3	4 MID	5	6	7	8	9 HIGH

Probe 2: low = 0, high = 3, mid = 1; arr[mid] = 18 ($18 > 5$) → search the left half

8	18	23	29	34	42	54	63	72	85
0 LOW	1 MID	2	3 HIGH	4	5	6	7	8	9

Probe 3: low = 0, high = 0, mid = 0; arr[mid] = 8 ($8 > 5$) → search the left half

8	18	23	29	34	42	54	63	72	85
0 LOW/MID/HIGH	1	2	3	4	5	6	7	8	9

PROBE	LOW	HIGH	MID	ARR[MID]	QUESTION	DECISION	CANDIDATES
1	0	9	$0 + (9 - 0) / 2 = 4$	34	$34 > 5$	search the left half	10
2	0	3	$0 + (3 - 0) / 2 = 1$	18	$18 > 5$	search the left half	4
3	0	0	$0 + (0 - 0) / 2 = 0$	8	$8 > 5$	search the left half	1
	0	-1			low > high	stop: return -1	0

3 probes and -1. The window empties from the right: high falls to -1, which is below low.

Larger than everything · search 100

Every probe says "go right".

8	18	23	29	34	42	54	63	72	85
0	1	2	3	4	5	6	7	8	9

Probe 1: low = 0, high = 9, mid = 4; arr[mid] = 34 (34 < 100) → search the right half

8	18	23	29	34	42	54	63	72	85
0 LOW	1	2	3	4 MID	5	6	7	8	9 HIGH

Probe 2: low = 5, high = 9, mid = 7; arr[mid] = 63 (63 < 100) → search the right half

8	18	23	29	34	42	54	63	72	85
0	1	2	3	4	5 LOW	6	7 MID	8	9 HIGH

Probe 3: low = 8, high = 9, mid = 8; arr[mid] = 72 (72 < 100) → search the right half

8	18	23	29	34	42	54	63	72	85
0	1	2	3	4	5	6	7	8 LOW/MID	9 HIGH

Probe 4: low = 9, high = 9, mid = 9; arr[mid] = 85 (85 < 100) → search the right half

8	18	23	29	34	42	54	63	72	85
0	1	2	3	4	5	6	7	8	9 LOW/MID/HIGH

PROBE	LOW	HIGH	MID	ARR[MID]	QUESTION	DECISION	CANDIDATES
1	0	9	$0 + (9 - 0) / 2 = 4$	34	$34 < 100$	search the right half	10
2	5	9	$5 + (9 - 5) / 2 = 7$	63	$63 < 100$	search the right half	5
3	8	9	$8 + (9 - 8) / 2 = 8$	72	$72 < 100$	search the right half	2
4	9	9	$9 + (9 - 9) / 2 = 9$	85	$85 < 100$	search the right half	1
	10	9			low > high	stop: return -1	0

4 probes and -1. This time low climbs to 10, past high.

Between two values · search 31

31 is not stored, but it would sit between 29 and 34. The same example the earlier section traced.

8	18	23	29	34	42	54	63	72	85
0	1	2	3	4	5	6	7	8	9

Probe 1: low = 0, high = 9, mid = 4; arr[mid] = 34 (34 > 31) → search the left half

8	18	23	29	34	42	54	63	72	85
0 LOW	1	2	3	4 MID	5	6	7	8	9 HIGH

Probe 2: low = 0, high = 3, mid = 1; arr[mid] = 18 (18 < 31) → search the right half

8	18	23	29	34	42	54	63	72	85
0 LOW	1 MID	2	3 HIGH	4	5	6	7	8	9

Probe 3: low = 2, high = 3, mid = 2; arr[mid] = 23 (23 < 31) → search the right half

8	18	23	29	34	42	54	63	72	85
0	1	2 LOW/MID	3 HIGH	4	5	6	7	8	9

Probe 4: low = 3, high = 3, mid = 3; arr[mid] = 29 (29 < 31) → search the right half

8	18	23	29	34	42	54	63	72	85
0	1	2	3 LOW/MID/HIGH	4	5	6	7	8	9

PROBE	LOW	HIGH	MID	ARR[MID]	QUESTION	DECISION	CANDIDATES
1	0	9	$0 + (9 - 0) / 2 = 4$	34	$34 > 31$	search the left half	10
2	0	3	$0 + (3 - 0) / 2 = 1$	18	$18 < 31$	search the right half	4
3	2	3	$2 + (3 - 2) / 2 = 2$	23	$23 < 31$	search the right half	2
4	3	3	$3 + (3 - 3) / 2 = 3$	29	$29 < 31$	search the right half	1
	4	3			low > high	stop: return -1	0

4 probes and -1. Not found is decided the moment the window is empty: no need to look at any more cells.

Exam-style question 3(c) · search 88 in {2, 6, 12, 32, 50, 59, 76, 83, 90}

"Searching for the number 88 will be determined after how many iterations?" 88 is not in the array, but it would fit between 83 and 90.

2	6	12	32	50	59	76	83	90
0	1	2	3	4	5	6	7	8

Probe 1: low = 0, high = 8, mid = 4; arr[mid] = 50 (50 < 88) → search the right half

2	6	12	32	50	59	76	83	90
0 LOW	1	2	3	4 MID	5	6	7	8 HIGH

Probe 2: low = 5, high = 8, mid = 6; arr[mid] = 76 (76 < 88) → search the right half

2	6	12	32	50	59	76	83	90
0	1	2	3	4	5 LOW	6 MID	7	8 HIGH

Probe 3: low = 7, high = 8, mid = 7; arr[mid] = 83 (83 < 88) → search the right half

2	6	12	32	50	59	76	83	90
0	1	2	3	4	5	6	7 LOW/MID	8 HIGH

Probe 4: low = 8, high = 8, mid = 8; arr[mid] = 90 (90 > 88) → search the left half

2	6	12	32	50	59	76	83	90
0	1	2	3	4	5	6	7	8 LOW/MID/HIGH

PROBE	LOW	HIGH	MID	ARR[MID]	QUESTION	DECISION	CANDIDATES
1	0	8	$0 + (8 - 0) / 2 = 4$	50	$50 < 88$	search the right half	9
2	5	8	$5 + (8 - 5) / 2 = 6$	76	$76 < 88$	search the right half	4
3	7	8	$7 + (8 - 7) / 2 = 7$	83	$83 < 88$	search the right half	2
4	8	8	$8 + (8 - 8) / 2 = 8$	90	$90 > 88$	search the left half	1
	8	7			low > high	stop: return -1	0

4 iterations, then the window is empty (`low = 8 > high = 7`) and the search returns `-1`. Each probe moves `low` up: the key is larger than 50, 76 and 83, but smaller than 90.

Searching / Binary search · Common mistakes

Common mistakes

- 1 Searching an unsorted array.**
Binary search on `[5, 1, 9, 3, 7]` for 3 (which is there, at index 3) returns `-1`: the first probe reads 9, concludes “3 must be to the left”, and discards the cell holding 3. Always check the precondition.
- 2 Computing `mid` as `(low + high) / 2`.**
Works for small arrays but can overflow for very large ones (see the earlier section). Write `low + (high - low) / 2`.
- 3 Looping with `low < high` instead of `low ≤ high`.**
When one candidate remains, `low == high`, and the loop quits without checking it. On `[1, 3, 5, 7, 9]`, searching for 9 (index 4) returns `-1` with the wrong condition.
- 4 Setting `low = mid` or `high = mid`.**
The window may stop shrinking and the loop never ends. Use `mid + 1` and `mid - 1`.
- 5 Getting the direction wrong for descending data.**
Larger-than-key means *go right* in a descending array. Decide this before you trace.
- 6 Comparing Strings with `<` or `==`.**
Use `compareTo` for order and `.equals` for equality.
- 7 Returning `mid` when the key is absent.**
Return `-1` only after the loop ends.
- 8 Miscalculating iterations.**
One iteration is one probe. The final check that finds `low > high` is not a probe.

Searching / Binary search · A test-ready summary

A test-ready summary

TEST ANSWER

Binary search finds a key in a sorted array. It keeps a window low...high, probes the middle at $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$, and either returns mid if `arr[mid]` equals the key or discards the half that cannot contain the key ($\text{low} = \text{mid} + 1$ or $\text{high} = \text{mid} - 1$). When the window is empty ($\text{low} > \text{high}$) it returns -1. Each probe halves the candidates, so the worst case is $O(\log n)$, at most $\lfloor \log_2 n \rfloor + 1$ probes, with $O(1)$ extra space. It requires sorted data and random access, and is far faster than linear search, $O(n)$, on large sorted arrays.

Searching / Binary search · Practice

Practice

Guided question

Use binary search to find 29 in [8, 18, 23, 29, 34, 42, 54, 63, 72, 85]. Write low, high, mid and `arr[mid]` for each probe.

▼ Reveal the guided answer

Probe 1: low = 0, high = 9, mid = 4; `arr[mid]` = 34 → search the left half

Probe 2: low = 0, high = 3, mid = 1; `arr[mid]` = 18 → search the right half

Probe 3: low = 2, high = 3, mid = 2; `arr[mid]` = 23 → search the right half

Probe 4: low = 3, high = 3, mid = 3; `arr[mid]` = 29 → found

Result: found at index 3 after 4 probes.

Independent question

Search for 50 in [3, 9, 14, 21, 35, 47, 60, 71]. How many iterations are needed, and what is returned?

- ☐ Start with `low = 0` and `high = 7`.
- ☐ Compute `mid` each time with $\text{low} + (\text{high} - \text{low}) / 2$.
- ☐ Decide left or right, and update `low` or `high`.
- ☐ Stop when the key is found or $\text{low} > \text{high}$.

▼ Reveal the independent answer

PROBE	LOW	HIGH	MID	ARR[MID]	QUESTION	DECISION	CANDIDATES
1	0	7	$0 + (7 - 0) / 2 = 3$	21	$21 < 50$	search the right half	8
2	4	7	$4 + (7 - 4) / 2 = 5$	47	$47 < 50$	search the right half	4
3	6	7	$6 + (7 - 6) / 2 = 6$	60	$60 > 50$	search the left half	2
	6	5			low > high	stop: return -1	0

3 iterations, returns -1: 50 is not in the array.

Test-style question

For the sorted array [2, 6, 12, 32, 50, 59, 76, 83, 90] of the past paper, after how many iterations will the search for 5 be decided, and what is returned? Then state the worst-case time complexity of binary search.

▼ Reveal the test-style answer

PROBE	LOW	HIGH	MID	ARR[MID]	QUESTION	DECISION	CANDIDATES
1	0	8	$0 + (8 - 0) / 2 = 4$	50	$50 > 5$	search the left half	9
2	0	3	$0 + (3 - 0) / 2 = 1$	6	$6 > 5$	search the left half	4
3	0	0	$0 + (0 - 0) / 2 = 0$	2	$2 < 5$	search the right half	1
	1	0			low > high	stop: return -1	0

3 iterations, returns -1. The worst case is **$O(\log n)$** .

Searching / Chapter summary

Choose the search from the data you have

QUESTION	LINEAR SEARCH	BINARY SEARCH
Must data be sorted?	No	Yes
Worst time	$O(n)$	$O(\log n)$

QUESTION	LINEAR SEARCH	BINARY SEARCH
Core movement	Advance one index	Discard half
Not-found condition	End of array	low > high

▼ Quick check: which search for one lookup in a small unsorted array?

Linear search. Sorting first would add work and could change the original order.

More worked cases: edge cases, duplicates and descending order

1. Absent between values

Input [8, 18, 23, 29, 34, 42, 54, 63, 72, 85]; key 31.

8	18	23	29	34	42	54	63	72	85
0	1	2	3	4	5	6	7	8	9

Final probe/check at index 3; return -1.

PROBE	LOW	HIGH	MID	VALUE	EVALUATED CONDITIONS AND UPDATE
1	0	9	4	34	34 == 31 → false; 34 < 31 → false; high = 3
2	0	3	1	18	18 == 31 → false; 18 < 31 → true; low = 2
3	2	3	2	23	23 == 31 → false; 23 < 31 → true; low = 3
4	3	3	3	29	29 == 31 → false; 29 < 31 → true; low = 4

Result -1; 4 midpoint probes.

2. First value

Input [8, 18, 23, 29, 34, 42, 54, 63, 72, 85]; key 8.



Final probe/check at index 0; return 0.

PROBE	LOW	HIGH	MID	VALUE	EVALUATED CONDITIONS AND UPDATE
1	0	9	4	34	$34 == 8 \rightarrow \text{false}$; $34 < 8 \rightarrow \text{false}$; $\text{high} = 3$
2	0	3	1	18	$18 == 8 \rightarrow \text{false}$; $18 < 8 \rightarrow \text{false}$; $\text{high} = 0$
3	0	0	0	8	$8 == 8 \rightarrow \text{true}$; return 0

Result 0; 3 midpoint probes.

3. Duplicate key

Input [10, 20, 20, 20, 30]; key 20.



Final probe/check at index 2; return 2.

PROBE	LOW	HIGH	MID	VALUE	EVALUATED CONDITIONS AND UPDATE
1	0	4	2	20	$20 == 20 \rightarrow \text{true}$; return 2

Result 2; 1 midpoint probes.

4. Descending order

Input [85, 72, 63, 54, 42, 34, 29, 23, 18, 8]; key 18; descending comparisons.



Final probe/check at index 8; return 8.

PROBE	LOW	HIGH	MID	VALUE	EVALUATED CONDITIONS AND UPDATE
1	0	9	4	42	$42 == 18 \rightarrow \text{false}$; $42 > 18 \rightarrow \text{true}$; low = 5
2	5	9	7	23	$23 == 18 \rightarrow \text{false}$; $23 > 18 \rightarrow \text{true}$; low = 8
3	8	9	8	18	$18 == 18 \rightarrow \text{true}$; return 8

Result 8; 3 midpoint probes.

5. Empty and singleton

Input [18]; key 31.

18

0

Final probe/check at index 0; return -1.

PROBE	LOW	HIGH	MID	VALUE	EVALUATED CONDITIONS AND UPDATE
1	0	0	0	18	$18 == 31 \rightarrow \text{false}$; $18 < 31 \rightarrow \text{true}$; low = 1

Result -1; 1 midpoint probes. For [] the loop/body is never entered and -1 is returned with zero probes.

Exam-style practice with worked answers

▼ A. Trace key 31 on the main input. Show every index/bound and the failure result. [6]

PROBE	LOW	HIGH	MID	VALUE	EVALUATED CONDITIONS AND UPDATE
1	0	9	4	34	$34 == 31 \rightarrow \text{false}$; $34 < 31 \rightarrow \text{false}$; $\text{high} = 3$
2	0	3	1	18	$18 == 31 \rightarrow \text{false}$; $18 < 31 \rightarrow \text{true}$; $\text{low} = 2$
3	2	3	2	23	$23 == 31 \rightarrow \text{false}$; $23 < 31 \rightarrow \text{true}$; $\text{low} = 3$
4	3	3	3	29	$29 == 31 \rightarrow \text{false}$; $29 < 31 \rightarrow \text{true}$; $\text{low} = 4$

Every retained candidate is eliminated; return -1. Four midpoint probes are required.

▼ B. Explain the best and worst time using input examples. [4]

Best: key equals the first midpoint, one probe. Worst: key absent or only located after the range repeatedly halves, logarithmic probes.

▼ C. Debug the bounds and return statement. [4]

low and high are inclusive. Use $\text{low} \leq \text{high}$, calculate the overflow-safe midpoint, and exclude mid using ± 1 . The not-found return belongs after the range empties.

▼ D. Return the first occurrence of a duplicate. What must change? [3]

Keep a result variable. On equality record mid, set $\text{high} = \text{mid} - 1$, and continue. Return the recorded index after the loop.

▼ E. Empty array: can this code access index 0? [2]

No. high is -1, so low <= high is false. It returns -1 without reading a cell.

Topic quiz: test your understanding

Answer all 12 questions. Choose an option to see the reasoning; reset to retry. The printable answer key follows the questions.

1. What is the last valid index of an array of length 6?

- ☐ A. 5
- ☐ B. 6
- ☐ C. 7

2. What does $O(1)$ auxiliary space describe?

- ☐ A. Exactly one machine instruction
- ☐ B. The input contains one value
- ☐ C. Extra storage stays bounded as n grows

3. What is a loop invariant?

- ☐ A. A loop that never terminates
- ☐ B. A statement that remains true at a specified loop boundary
- ☐ C. A variable that never changes

4. What precondition makes discarding half valid?

- ☐ A. The range is sorted by the searched key in the expected direction
- ☐ B. The key is positive
- ☐ C. There are no duplicates

5. $\text{low}=0$, $\text{high}=9$. What midpoint does $\text{low}+(\text{high}-\text{low})/2$ give?

- ☐ A. 5
- ☐ B. 9
- ☐ C. 4

6. Ascending data: $\text{arr}[\text{mid}] < \text{key}$. What update follows?

- ☐ A. $\text{low} = \text{mid}$
- ☐ B. $\text{low} = \text{mid} + 1$
- ☐ C. $\text{high} = \text{mid} - 1$

7. Ascending data: $\text{arr}[\text{mid}] > \text{key}$. What update follows?

- ☐ A. $\text{high} = \text{mid} - 1$
- ☐ B. $\text{low} = \text{mid} + 1$
- ☐ C. $\text{high} = \text{mid}$

8. Why use $\text{low} \leq \text{high}$ in the loop?

- ☐ A. It allows indexes past the array
- ☐ B. It guarantees the key exists
- ☐ C. A one-cell candidate range still needs checking

9. Search the opening map for 7. Which midpoint indexes are probed?

- ☐ A. 4, 8, 9
- ☐ B. 4, 7, 8
- ☐ C. 5, 8

10. Does ordinary binary search guarantee the first duplicate?

- ☐ A. No, it returns a matching midpoint
- ☐ B. Yes
- ☐ C. Duplicates are illegal

11. Worst-case time for random-access sorted data?

- ☐ A. $O(n^2)$
- ☐ B. $O(1)$
- ☐ C. $O(\log n)$

12. Descending data: $\text{arr}[\text{mid}] > \text{key}$. Which side remains possible?

- ☐ A. Both halves must be retained
- ☐ B. Right; set $\text{low} = \text{mid} + 1$
- ☐ C. Left; set $\text{high} = \text{mid} - 1$

▼ Answer key and explanations

1. **5.** Indexes start at 0, so the last index is $\text{length} - 1$.
2. **Extra storage stays bounded as n grows.** Auxiliary space counts extra working storage, excluding the input array.
3. **A statement that remains true at a specified loop boundary.** The invariant explains why each iteration preserves progress toward a correct result.
4. **The range is sorted by the searched key in the expected direction.** Only consistent ordering proves that one entire half cannot contain the key.
5. **4.** Integer division floors the nonnegative offset: $9 / 2$ is 4.
6. **$\text{low} = \text{mid} + 1$.** The midpoint and every smaller value to its left are ruled out.
7. **$\text{high} = \text{mid} - 1$.** The midpoint has failed and must be excluded along with the too-large right side.
8. **A one-cell candidate range still needs checking.** $\text{low} == \text{high}$ is one remaining candidate; $\text{low} > \text{high}$ is the empty range.
9. **4, 7, 8.** The inclusive windows are 0..9, 5..9, then 8..9.
10. **No, it returns a matching midpoint.** Finding the first occurrence requires continuing left after a match.
11. **$O(\log n)$.** Each unsuccessful probe roughly halves the remaining candidates.
12. **Right; set $\text{low} = \text{mid} + 1$.** Smaller values lie to the right in descending order.